Retrieval X-Ray: A Hybrid Neural Retrieval System with Incremental Maintenance and Integrated Debugging

Piyush Mittal
Department of Computer Science and Engineering
Maharaja Agrasen Institute of Technology (MAIT)
Rohini, New Delhi, India

Dr. Ankita Gupta
Project Mentor and Assistant Professor
Department of Computer Science and Engineering
Maharaja Agrasen Institute of Technology (MAIT)
Rohini, New Delhi, India

***Abstract*—This paper presents Retrieval X-Ray, a Rust-based hybrid retrieval system for computer science content that combines web crawling, content normalization, sentence-aware chunking, dense embedding, lexical indexing, and server-side result delivery within a single end-to-end architecture. The system is designed not only to retrieve relevant passages, but also to make retrieval behavior inspectable through score decomposition and dedicated debugging interfaces. To support a mutable corpus under constrained hardware, the architecture uses RocksDB-backed persistent state, page-state manifests, content-hash chunk identifiers, queue-driven downstream maintenance, vector tombstones, and a composite vector backend consisting of a compacted Hierarchical Navigable Small World (HNSW) base index with a mutable brute-force delta overlay. Across the project, two findings were especially important: first, corpus engineering and representation design affected retrieval quality as strongly as model or ranker changes; second, local memory ceilings materially shaped the final architecture, motivating incremental maintenance and remote rebuild workflows. The paper details the system design, implementation decisions, and observed engineering trade-offs, and argues that debuggability should be treated as a first-class property of modern retrieval systems.**

***Index Terms*—information retrieval, neural retrieval, hybrid retrieval, HNSW, incremental indexing, RocksDB, retrieval debugging**

I. Introduction

Neural retrieval systems now play a central role in question answering, developer tooling, and retrieval-augmented generation (RAG) pipelines [1]. In practice, however, building a useful retriever requires much more than selecting an embedding model. Real systems must operate over noisy corpora, handle incremental updates, maintain multiple derived indexes, remain debuggable under ranking failures, and fit within concrete hardware limits. These concerns become especially visible in domain-focused search, where relevance depends on terminology, document structure, source quality, and the interaction between lexical and semantic signals.

This paper presents Retrieval X-Ray, a hybrid neural retrieval system built in Rust for computer science content. The system crawls and normalizes technical pages, produces retrieval-oriented chunks, indexes them with both dense and lexical methods, serves hybrid-ranked results, and exposes ranking behavior through debugging surfaces designed for retrieval analysis. The resulting artifact is both a functioning search system and a case study in retrieval systems engineering under constrained local resources.

Computer science content is an instructive retrieval domain. Queries often mix exact terminology, abbreviations, and conceptual phrasing; relevant evidence may appear in headings, list items, table rows, or short explanatory sentences whose meaning depends on surrounding context. Purely lexical retrieval can miss semantically relevant passages because of vocabulary mismatch, while purely dense retrieval can surface topically related but operationally unhelpful results. A hybrid architecture is therefore natural, but hybrid systems also demand stronger observability if they are to be improved systematically.

A second motivation comes from the needs of modern RAG systems. In many applications, the key question is not merely whether some plausible passages are returned, but whether developers can inspect failures, understand which signals dominated ranking, and determine whether the problem lies in chunking, representation, candidate generation, or reranking. Aggregate metrics are necessary but insufficient; per-result visibility is equally important.

The system design in this work was also shaped by practical constraints. University-scale and individual projects rarely have access to industrial search infrastructure. As a result, the architecture must favor durability, resumability, bounded memory use, and targeted offloading of heavyweight rebuild tasks. In Retrieval X-Ray, those constraints are not treated as incidental deployment details; they are treated as part of the retrieval problem itself.

This paper makes four contributions. First, it presents an end-to-end Rust retrieval system for computer science content with crawling, normalization, chunking, embedding, hybrid search, and serving. Second, it describes an incremental maintenance architecture based on page-state manifests, content-hash chunk identifiers, queue-driven index updates, and a base-plus-delta vector design. Third, it reports engineering findings about the strong effect of corpus curation and representation design on retrieval quality. Fourth, it demonstrates a retrieval debugging approach centered on score decomposition and an interactive terminal interface, supporting the broader claim that retrieval

systems should be designed to be inspectable as well as accurate.

II. System Design

A. Overview

Retrieval X-Ray follows a staged retrieval pipeline:

1) crawl web pages,
2) normalize and extract structured text,
3) chunk pages into retrieval units,
4) embed chunks with a dense model,
5) build or update vector and lexical indexes,
6) serve ranked results,
7) inspect retrieval behavior through debugging tools.

The pipeline is backed by RocksDB, which acts as the durable source of truth for corpus state and intermediate artifacts. Derived indexes are treated as rebuildable views over persistent page, chunk, and embedding state rather than as the primary store of record [4].

B. Persistent Storage in RocksDB

The system stores operational and retrieval data in RocksDB using multiple column families. The schema includes families for seen URLs, crawl frontier, domains and robots metadata, normalized page content, chunks, embeddings, click logs, normalization queue, page-state metadata, embedding queue, vector queue, lexical queue, and vector tombstones.

This organization separates durable corpus content from derived retrieval artifacts and mutable maintenance state. Instead of treating the retrieval system as a monolithic index, Retrieval X-Ray stores enough structure to rerun downstream stages independently and to localize updates to the affected pages and chunks.

C. Page-State Manifests and Incremental Maintenance

To support a mutable corpus, the system maintains an explicit `page_state` record for each page. This record stores a content hash, the set of chunk identifiers derived from the current page version, crawl and fetch timestamps, and optional HTTP metadata such as ETag and Last-Modified.

Chunk identifiers are generated from the tuple (`URL`, `content_hash`, `position`) rather than from simple positional numbering. When a page changes, newly generated chunks receive new identifiers, and obsolete chunks can be identified precisely.

This design enables queue-driven downstream maintenance. The system uses a `normalize_queue` for pages awaiting chunk production, an `embed_queue` for chunks awaiting embedding, a `vector_queue` for vector upserts and deletes, a `lexical_queue` for lexical upserts and deletes, and `vector_tombstones` to hide deleted vectors until compaction or rebuild. As a result, corpus mutation becomes a sequence of explicit state transitions rather than a trigger for immediate full rebuilds.

D. Chunking and Text Representation

The retrieval unit is a chunk derived from normalized page blocks. Chunking is sentence-oriented but not restricted to isolated single sentences. A heuristic sentencizer handles common false boundaries such as abbreviations, decimal numbers, URLs, ellipses, and initials. Sentences are then merged into sliding windows; the default configuration uses a window size of 3 with overlap 1.

The system distinguishes between display text, used for presentation, and embedding text, used for dense representation. Embedding text is deliberately richer and may include the page title, heading chain, and chunk body text. This distinction is important in technical retrieval because many relevant snippets are short and become substantially more informative when paired with their heading or document context.

The implementation also preserves a notion of statement chaining. Earlier work explored a training pipeline for chaining-related classification, but the production path remained heuristic. The current system therefore uses heuristic chaining logic to preserve antecedent context where possible, and non-leaf chunks can be filtered from final retrieval.

E. Dense Retrieval

The dense retriever uses the `bge-small-en-v1.5` model with 384-dimensional embeddings [5], [6]. Query encoding applies model-appropriate formatting for BGE-style retrieval prompts, and vectors are L2-normalized for cosine similarity.

Embedding is implemented in two paths: an online query path through a singleton embedding client, and a bulk path using direct ONNX Runtime sessions for higher-throughput offline processing. The repository is configured to use CoreML on macOS and CUDA on non-macOS platforms when available.

F. Vector Index Design

The vector backend uses a composite design with two layers: a compacted HNSW base snapshot for most retrieval and a mutable brute-force delta overlay for recent updates [2]. The delta layer allows newly embedded chunks to become searchable immediately without requiring a full HNSW rebuild. Deletions are represented through tombstones and are filtered at query time.

This design reflects a practical compromise. HNSW offers scalable approximate nearest-neighbor search at corpus scale, but full rebuilds are expensive and memory-intensive. The brute-force delta is less efficient but easy to update incrementally and effective when the mutable frontier remains relatively small.

G. Lexical Retrieval and Hybrid Fusion

Lexical retrieval is implemented with Tantivy and follows BM25-style ranking principles [3]. The lexical index stores fields including chunk identifier, title, section, body text, heading, and source URL. Titles and headings are boosted more strongly than body text, especially for short technical queries, and phrase-oriented clauses are supported for compact terminology-heavy searches.

Hybrid retrieval proceeds in two stages. First, vector and lexical search produce candidate sets independently. Second, the candidates are combined using reciprocal rank fusion and reranked with lightweight features.

The reranker incorporates normalized vector score, normalized lexical score, token overlap with title, heading, and body fields, exact phrase matches, and a domain-authority bonus for official sources. The query pipeline also includes hand-authored synonym expansion for common technical abbreviations and paired terms such as `js/javascript`, `ts/typescript`, `auth/authentication`, and `gpu/cuda`.

H. Serving and Debugging

The serving layer uses Axum to provide server-side rendered search results and click tracking backed by RocksDB. In parallel, the system provides a retrieval debugging interface implemented as a terminal user interface (TUI) using `ratatui` and `crossterm`. The TUI exposes query input, ranked results, detailed score inspection, strategy comparison, and failure-analysis views. Each result carries decomposed ranking features, making it possible to inspect not only the final rank but also the signals that produced it.

This debugging layer is central to the system's design philosophy. Retrieval X-Ray is intended not only to retrieve passages, but also to support direct analysis of why those passages were returned.

III. Implementation

A. Crawling, Canonicalization, and Extraction

The crawler implements URL canonicalization, DNS filtering, robots handling, HTML fetching, extraction, and link discovery. Early milestones established HTTPS-only guards, host normalization, query sorting, trailing-slash deduplication, filtering of private and loopback addresses, robots caching in RocksDB, metadata extraction, density-based filtering, and heading-aware text normalization.

A significant architectural decision was to separate crawling from heavy downstream processing. Rather than generating chunks and maintaining retrieval artifacts directly inside the crawl hot path, the system persists page-level content first and performs normalization and indexing through downstream queues. This reduces coupling between fetch latency and retrieval maintenance and makes reprocessing possible without re-crawling.

B. Normalization and Chunk Production

The `normalize_pages` stage reads page records from RocksDB, computes a content hash, and compares it against stored page state. If content is unchanged, the page is skipped. If content has changed, the stage regenerates chunks, writes new chunk records, removes obsolete chunk and embedding entries, and enqueues downstream vector and lexical maintenance work.

This stage is one of the system's core design elements. Instead of treating indexing as a blind rebuild process, the system tracks which chunks correspond to which page version and updates only the affected downstream artifacts.

C. Embedding Pipeline

The current embedding configuration uses `bge-small-en-v1.5` at 384 dimensions. The bulk pipeline relies on direct ONNX Runtime sessions and explicit tokenizer control to achieve better throughput than the simpler online path. Chunks are embedded from `embed_text`, which incorporates page title and heading context when available.

Configuration values were tuned for local safety as well as speed. The main local configuration uses batch size 64, maximum input length 128, one bulk worker, and two intra-operation threads per worker. These settings were chosen conservatively for a local GTX 1660 Super with 6 GB of VRAM.

D. Vector and Lexical Indexing

The vector indexing implementation supports both full rebuilds and incremental updates. Full rebuilds stream embeddings from RocksDB into an HNSW index. Incremental updates are applied to the brute-force delta layer through the vector queue, while deletions are reflected in tombstones.

Lexical indexing uses Tantivy and supports both rebuild and update workflows. Because the lexical schema retains structural fields such as title and section, ranking can use document structure explicitly rather than relying only on dense embeddings.

At runtime, the search stack consists of a RocksDB database, a vector index composed of an HNSW base plus an optional brute-force delta, and an optional lexical index. This architecture keeps the database as the authoritative corpus and treats all indexes as derived structures.

E. Evaluation Utilities

The repository includes evaluation utilities for qrels-based measurement, with support for Mean Reciprocal Rank, NDCG@k, and Recall@k. It also includes benchmark binaries for query latency, approximate nearest-neighbor benchmarking, and embedding throughput.

These utilities are functionally useful, but the broader evaluation suite remains incomplete. Accordingly, they should be understood as supporting infrastructure rather than as a finalized experimental framework.

F. Retrieval Debugging TUI

The TUI is implemented in `src/tui/` with separate application state, input handlers, and rendering logic, and is exposed through `debug_tui` and `debug_tui_integrated` binaries. The interface supports multiple inspection views and displays per-result features such as final score, vector score, lexical score, title overlap, heading overlap, body overlap, and authority bonus.

This design makes debugging part of the retrieval stack itself rather than an afterthought delegated to logs or offline scripts.

TABLE I
CORPUS SCALE ACROSS MAJOR PROJECT PHASES.

Phase	Pages	Chunks	Embeddings
Earlier large snapshot	93,481	4,946,045	4,654,025
Post-pruning snapshot	61,431	3,588,853	3,367,309
High-quality profile	80,801	4,507,321	4,243,078

G. Engineering Evolution

The repository history reflects a staged progression from crawl and extraction utilities to a full hybrid retrieval system. Early milestones focused on crawler correctness and content extraction, followed by sentence chunking and context utilities, then embedding, vector indexing, query execution, and finally search serving and debugging interfaces.

A major redesign occurred when corpus growth made naive rebuild assumptions increasingly expensive. The introduction of `page_state`, content-hash chunk identifiers, downstream maintenance queues, and vector tombstones marked a transition from a static-corpus mindset to a mutable-corpus architecture. This redesign enabled re-normalization without re-crawling, precise deletion of obsolete chunks, separation of page mutation from index mutation, and fresher incremental updates without constant full rebuilds.

The project also showed that retrieval quality gains came from representation and corpus choices at least as much as from model selection. Repeatedly useful changes included richer `embed_text` construction, sliding-window chunking, synonym expansion for technical abbreviations, title and heading overlap features, phrase signals, and authority-sensitive bonuses for official domains. The broader lesson was that technical retrieval quality depends strongly on representing structure explicitly.

IV. RESULTS AND OBSERVATIONS

This section reports grounded observations from the implemented system and available project measurements. It does not claim a fully completed final benchmark suite.

A. Corpus Scale

Across different stages, the system operated at multi-million-chunk scale. Table I summarizes major corpus snapshots.

The corpus scale places the system well beyond toy settings and helps explain why incremental maintenance, queue design, and rebuild workflows became central architectural concerns.

B. Embedding Throughput

The embedding pipeline improved substantially over the course of the project. Table II summarizes recovered throughput and system-constraint milestones.

On Apple Silicon, throughput improved from roughly 33 embeddings per second to about 61 after batching and pipeline tuning. On a local GTX 1660 Super, a conservative CUDA configuration achieved approximately 509 to 522 embeddings per second. These gains show that session management, batching, sequence length, and backend selection had major practical impact. They also show that faster embedding alone did not remove the index-build bottleneck.

TABLE II
RECOVERED PERFORMANCE AND CONSTRAINT MILESTONES.

Metric	Value	Context
Apple Silicon baseline	∼33 emb/s	Early M2 CPU embedding path
Apple Silicon optimized	∼61 emb/s	After batching and pipeline tuning
Local GPU embedding	509–522 emb/s	GTX 1660 Super under safer local settings
HNSW memory wall	∼9.7 GB RSS	Local full rebuild OOM on 16 GB machine
Remote query latency	1.6–2.0 s	Example CUDA query path on Modal

C. Memory Limits as a First-Class Result

One of the clearest systems findings was the repeated failure of full local HNSW rebuilds at roughly 9.7 GB RSS on a 16 GB machine. This observation was not merely operational; it materially influenced the architecture. It validated the need for queue-based incremental updates, the base-plus-delta vector design, and a remote rebuild workflow for heavyweight maintenance.

In other words, the final architecture was shaped by observed memory ceilings rather than by abstract design preference.

D. Retrieval Quality Observations

The available evidence on retrieval quality is more qualitative than quantitative, but several conclusions are well supported by implementation history and repeated testing.

First, richer embedding text improved retrieval. Including page-title and heading context made many technical passages more retrievable semantically. Second, chunking strategy mattered substantially: sentence-only chunking, paragraph-style grouping, and sliding windows behaved differently, especially for explanations distributed across adjacent sentences. Third, hybrid reranking improved short technical queries, with phrase matches, field overlap, and official-domain bonuses proving especially useful. Fourth, corpus quality often dominated ranker tweaks; removing noisy hosts and isolating a higher-quality profile had outsized effect.

Earlier pilot-style evaluation on a limited internal setup also suggested that chunking choice strongly affects effectiveness. In that setting, sentence chunking reached approximately MRR 0.71 and Recall@3 0.62, paragraph chunking reached approximately MRR 0.83 and Recall@3 0.78, and a fixed-512 baseline reached approximately MRR 0.64 and Recall@3 0.54. These figures should be interpreted as preliminary internal evidence rather than as final benchmark claims, but they are directionally informative and support the need for dedicated comparison and debugging tools.

E. Debuggability as an Outcome

A distinctive outcome of the project is that retrieval inspection became part of the system's identity rather than a secondary utility. The TUI exposes per-hit score decomposition instead of only final rankings. This supports workflows such as identifying cases where lexical evidence is strong but semantic evidence is weak, diagnosing heading mismatch or context fragmentation, comparing ranking strategies, and verifying whether authority or phrase signals helped or hurt.

Even without a fully developed browser-based analysis dashboard, the system demonstrates that retrieval quality work benefits from observability tools designed directly into the ranking pipeline.

V. Limitations

Retrieval X-Ray remains a substantial prototype rather than a finalized production search platform, and several limitations should be stated explicitly.

First, the evaluation story is incomplete. The repository includes metric computation and benchmark utilities, but larger recall studies, broader query-set evaluation, and report-grade benchmark tables remain unfinished. The available quantitative evidence is therefore partial.

Second, the latest verified debugging interface is a TUI rather than a full browser-based analysis environment. The TUI is useful and meaningful, but some richer debugging workflows remain future work.

Third, full vector maintenance still depends on occasional expensive rebuilds. The base-plus-delta design reduces that pressure, but does not eliminate it. Large-scale HNSW compaction remains costly enough to require remote infrastructure on the available local hardware.

Fourth, parts of the ranking logic are heuristic and domain-specific. Synonym expansion, authority bonuses, and short-query weighting improve practical relevance, but they are hand-authored rather than learned and may not generalize across domains.

Fifth, the corpus remains web-derived and therefore noisy by nature. Strong improvements were achieved through host pruning and profile isolation, but corpus quality remains an ongoing systems challenge rather than a solved preprocessing step.

Finally, earlier experimental artifacts around learning-based statement chaining were not fully integrated into the production retrieval path. Runtime chaining behavior is therefore still heuristic.

VI. Future Work

Several directions follow naturally from the current system.

The most immediate need is a more rigorous evaluation program: a stable benchmark query set, explicit qrels coverage, repeated comparison across chunking strategies, controlled testing of retrieval configurations, and latency and recall analysis under incremental versus rebuilt indexes. This would convert current observations into stronger experimental claims.

The TUI provides a solid foundation for retrieval inspection, but it could be extended with live integration against all active retrieval modes, annotation and qrels export, comparative A/B ranking analysis, and saved debugging sessions or report generation. This would strengthen the X-Ray component of the system from a prototype interface into a more systematic retrieval workbench.

Given how strongly corpus engineering affected outcomes, future work should also prioritize better host-quality scoring, automated low-signal path suppression, stronger seed and profile management, more explicit freshness and recrawl policies, and source-class-aware corpora separating official documentation, tutorials, and encyclopedic material.

On the indexing side, further work could explore automated thresholds for delta compaction, stronger lexical incremental maintenance, alternative approximate nearest-neighbor backends or build strategies, and more effective remote-local orchestration under memory constraints.

Because the system already records retrieval decomposition, a natural extension is to connect retrieval inspection to downstream generation quality. This would support analysis of how retrieval failures propagate into RAG outputs, enabling end-to-end grounded-answer debugging rather than passage-ranking inspection alone.

VII. Conclusion

Retrieval X-Ray demonstrates that building a neural retrieval system is as much a systems engineering problem as a ranking problem. The project produced a working Rust-based hybrid retrieval engine for computer science content, with persistent RocksDB storage, chunking and embedding pipelines, HNSW-based dense retrieval, Tantivy-based lexical retrieval, hybrid reranking, and result serving. More importantly, it evolved toward an architecture that treats mutation, rebuild cost, and debuggability as core design concerns.

Three conclusions stand out. First, dense retrieval quality depended strongly on representation choices such as heading context, page-title context, and chunk boundaries. Second, corpus engineering through host pruning, domain caps, and isolated higher-quality profiles was one of the strongest determinants of retrieval quality. Third, resource constraints were not peripheral: the observed failure of full local HNSW rebuilds at around 9.7 GB RSS directly motivated the system's incremental design and remote rebuild workflow.

The final framing around Retrieval X-Ray is therefore appropriate. The system is not only a hybrid search engine, but also an attempt to make retrieval behavior visible. The implemented TUI, partial evaluation utilities, and score-decomposed ranking pipeline point toward a broader research direction: retrieval systems should be designed not only to rank documents well, but also to explain themselves well enough to be improved systematically.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela,

"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[3] S. E. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[4] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du, "Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook," in *18th USENIX Conference on File and Storage Technologies (FAST)*, 2020.

[5] BAAI, "`bge-small-en-v1.5` Model Card," *Hugging Face*, 2023. [Online]. Available: https://huggingface.co/BAAI/bge-small-en-v1.5. [Accessed: Aug. 10, 2025].

[6] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," *arXiv preprint arXiv:2210.07316*, 2022.

[7] W. Lin, "Building a Web Search Engine from Scratch in Two Months with 3 Billion Neural Embeddings," Aug. 10, 2025. [Online]. Available: https://blog.wilsonl.in/search-engine/. [Accessed: Aug. 10, 2025].